

Linux Class 8 - Task

awk, Recursive Search and Working with a Data File

Submitted by: Shivakumar Muradi

Date: 30 September 2026

For this task I made a folder called class8. Inside it I typed the employees.txt file in vi, with the five rows, the same departments and the same salaries. I only changed the names to Indian ones while practising. For the search question I also made a small folder called docs and two or three plain text files, so that there was something to search through.

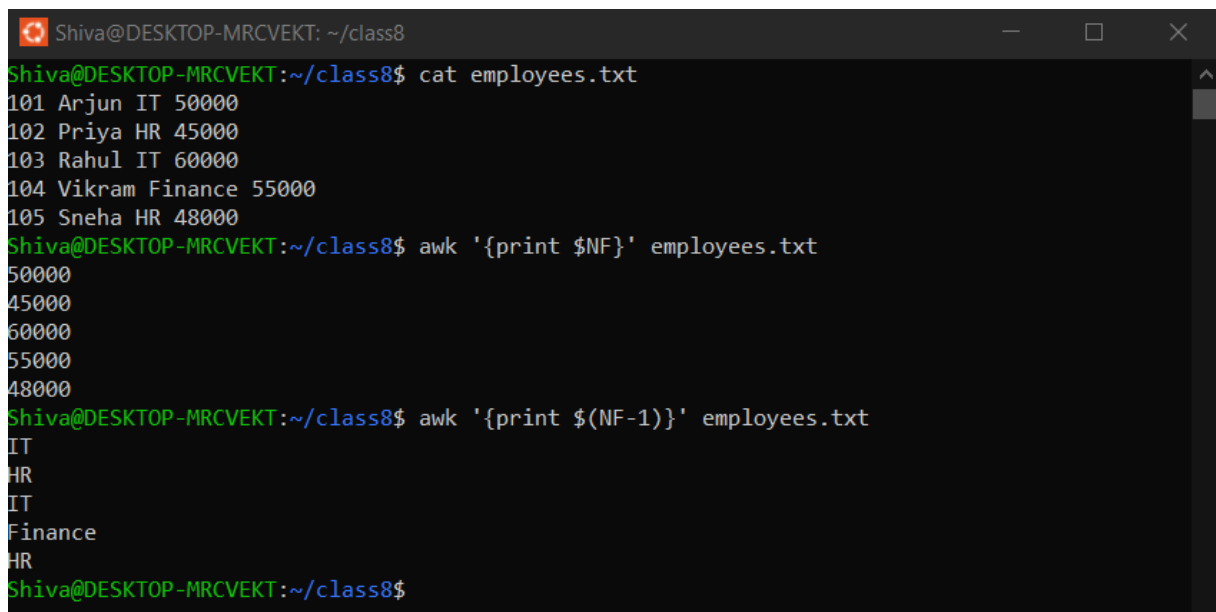
1. Which awk command would you use to print only the last column of each line?

The command is:

```
awk '{print $NF}' employees.txt
```

awk cuts every line into columns at the spaces, and then gives each column a number. \$1 is the first one, \$2 the second, and so on. NF is a special word that holds how many columns that line has, so \$NF means the last column, whatever number it happens to be.

The good thing about \$NF is that I do not have to count. In my file the salary is the 4th column, so awk '{print \$4}' would also work. But if one line had an extra column, \$4 would give the wrong thing while \$NF would still give the last one.



```
Shiva@DESKTOP-MRCVEKT: ~/class8
Shiva@DESKTOP-MRCVEKT:~/class8$ cat employees.txt
101 Arjun IT 50000
102 Priya HR 45000
103 Rahul IT 60000
104 Vikram Finance 55000
105 Sneha HR 48000
Shiva@DESKTOP-MRCVEKT:~/class8$ awk '{print $NF}' employees.txt
50000
45000
60000
55000
48000
Shiva@DESKTOP-MRCVEKT:~/class8$ awk '{print $(NF-1)}' employees.txt
IT
HR
IT
Finance
HR
Shiva@DESKTOP-MRCVEKT:~/class8$
```

Screenshot 1: the practice file, then the last column, then the column before it.

These are the ones I used:

I write	And awk gives me
\$1	The first column.
\$2	The second column, and so on.
\$0	The whole line, just as it is.
NF	How many columns this line has. It is a number, not a column.
\$NF	The last column, because NF is the number of the last one.
\$(NF-1)	The column just before the last one. The brackets are needed here.

I write	And awk gives me
NR	The line number awk is on.

The brackets in $\$(NF-1)$ are important. If I write $\$NF-1$ awk reads it as take the last column and subtract 1 from it, which gives a wrong answer. The brackets tell awk to work out $NF-1$ first and then take that column.

How an awk command is put together

Every awk command has the same shape:

```
awk 'pattern { action }' filename
```

- The pattern says which lines to work on. If I leave it out, awk works on every line.
- The action, inside the curly brackets, says what to do with those lines.
- The whole thing is kept inside single quotes so that the shell does not try to read the \$ signs itself.

So awk '{print \$NF}' has no pattern, which means every line, and the action is print the last column.

2. How do you search all files recursively and display only the filenames containing the word ABC?

The command is:

```
grep -r1 "ABC" .
```

- -r means recursive, so grep goes into every folder inside this one as well.
- -l means list the file names only, and not the matching lines.
- The word to look for goes in quotes.
- The dot at the end means start searching from the folder I am in.

To see the difference I ran it both ways. Without -l, grep printed the file name and the whole matching line. With -l it printed only the two file names, including the one inside the docs folder, which proves that -r really did go inside the folder.

```

Shiva@DESKTOP-MRCVEKT: ~/class8
Shiva@DESKTOP-MRCVEKT:~/class8$ ls -R
.:
docs employees.txt file1.txt file3.txt

./docs:
file2.txt
Shiva@DESKTOP-MRCVEKT:~/class8$ grep -r "ABC" .
./docs/file2.txt:This line has ABC in it.
./file1.txt:This line has ABC in it.
Shiva@DESKTOP-MRCVEKT:~/class8$ grep -r1 "ABC" .
./docs/file2.txt
./file1.txt
Shiva@DESKTOP-MRCVEKT:~/class8$

```

Screenshot 2: the same search with and without the -l option.

Other options that go with it:

Command	What it does
grep -r "ABC" .	Shows the matching lines, with the file name in front of each.
grep -r1 "ABC" .	Shows only the file names. This is the answer to the question.
grep -rli "abc" .	Same, but ignoring capital and small letters.

Command	What it does
<code>grep -rlw "ABC" .</code>	Only when ABC is a whole word, not part of a longer word.
<code>grep -rL "ABC" .</code>	The opposite: files that do NOT have the word.
<code>grep -rn "ABC" .</code>	Lines with the line number as well.

If I only want to know how many files have the word, I can count the names instead of reading them, with `grep -rl "ABC" .` and then counting the lines.

3. Explore the free and free -h command

`free` shows how much memory, that is RAM, the system has and how much of it is being used right now. It prints the numbers in kilobytes, which are hard to read, so there is an option to make them readable:

```
free
```

```
free -h
```

The `h` stands for human readable. The total on my machine is 8076928 in the first one and 7.7Gi in the second one, which is the same amount written in a way a person can understand.

```

Shiva@DESKTOP-MRCVEKT: ~/class8
Shiva@DESKTOP-MRCVEKT:~/class8$ free
              total        used        free      shared  buff/cache   available
Mem:           8076928       552792       7478244         3872        200884        7524136
Swap:          2097152           0         2097152
Shiva@DESKTOP-MRCVEKT:~/class8$ free -h
              total        used        free      shared  buff/cache   available
Mem:           7.7Gi         539Mi       7.1Gi         3.8Mi        196Mi        7.2Gi
Swap:          2.0Gi           0B         2.0Gi
Shiva@DESKTOP-MRCVEKT:~/class8$

```

Screenshot 3: the same memory shown in kilobytes and then in GB.

The columns mean:

- total is all the RAM in the machine.
- used is how much the programs are using.
- free is how much is completely empty.
- shared is memory that more than one program is using together.
- buff/cache is memory Linux is borrowing to keep recently used files handy.
- available is how much a new program could actually get.

The part that confused me is why free and available are different. Linux keeps old files in the buff/cache memory so it does not have to read them from the disk again. That memory looks busy, but if a new program needs it, Linux hands it over straight away. So available is the honest number, not free.

The second line is Swap. Swap is space on the hard disk used like extra memory when the RAM gets full. Mine shows 2.0Gi total and 0B used, so it has not been needed.

A few more options: `free -m` gives megabytes, `free -g` gives gigabytes, `free -t` adds a totals line at the bottom, and `free -s 2` keeps printing again every 2 seconds until I press Ctrl+C.

4. Questions on the employees.txt file

The file has four columns on every line: the id number, the name, the department and the salary. So \$1 is the id, \$2 is the name, \$3 is the department and \$4 is the salary. All four answers below use those numbers.

a) Display the name and salary of employees whose department is IT

```
awk '$3 == "IT" {print $2, $4}' employees.txt
```

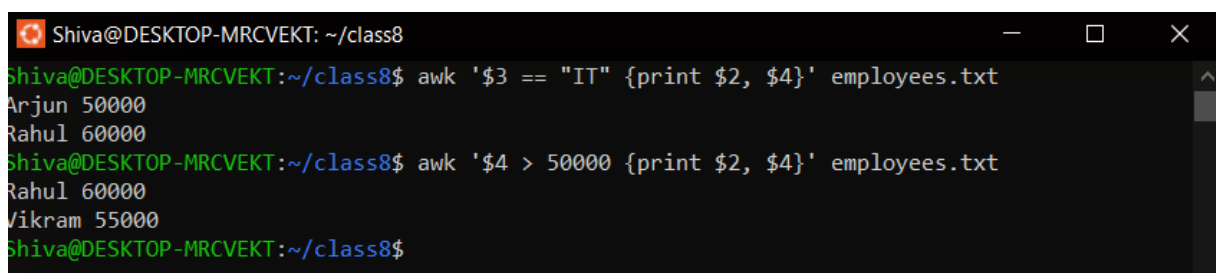
Here `$3 == "IT"` is the pattern. The two equal signs mean is equal to, and the word IT is in double quotes because it is text and not a number. awk checks every line, and only for the lines where the third column is exactly IT does it run the action and print the name and the salary.

The comma between `$2` and `$4` is what puts a space between them when they are printed. Without the comma the two would be stuck together.

b) Display the name and salary of employees earning more than 50000

```
awk '$4 > 50000 {print $2, $4}' employees.txt
```

This time the pattern compares numbers, so 50000 has no quotes around it. More than 50000 does not include 50000 itself, which is why Arjun does not come in this list even though his salary is exactly 50000. If I wanted him as well I would write `$4 >= 50000`.



```
Shiva@DESKTOP-MRCVEKT: ~/class8
Shiva@DESKTOP-MRCVEKT:~/class8$ awk '$3 == "IT" {print $2, $4}' employees.txt
Arjun 50000
Rahul 60000
Shiva@DESKTOP-MRCVEKT:~/class8$ awk '$4 > 50000 {print $2, $4}' employees.txt
Rahul 60000
Vikram 55000
Shiva@DESKTOP-MRCVEKT:~/class8$
```

Screenshot 4: the answers to a) and b).

c) Calculate the sum of all salaries

```
awk '{sum += $4} END {print sum}' employees.txt
```

This one has two parts. The first part has no pattern, so it runs on every line, and `sum += $4` means add this line's salary to a box called sum. I did not have to create the box or start it at zero, awk does that by itself.

END is a special pattern that means after the last line is finished. So the printing happens only once, at the very end, and not for every line. The answer was 258000.

d) Display the name and salary of the employee with the highest salary

There is no ready-made command for this, so I had to keep track of the biggest one myself as awk went down the file:

```
awk '$4 > max {max = $4; name = $2} END {print name, max}' employees.txt
```

It reads like this. For each line, if this salary is bigger than the biggest one I have seen so far, then remember this salary in max and this name in name. At the end, print them. The semicolon inside the brackets just separates the two things being remembered.

It works on the first line too, because max starts off empty, which awk treats as 0, and any salary is bigger than 0.

A simpler way without awk is to sort the file by the salary column:

```
sort -k4 -n employees.txt
```

-k4 means sort using the 4th column and -n means treat it as a number and not as text. The highest earner then comes out as the last line. This is the mistake I made the first time: without -n, sort compares them like words, and 9000 would come after 60000 because it starts with a 9.

```
Shiva@DESKTOP-MRCVEKT: ~/class8
Shiva@DESKTOP-MRCVEKT:~/class8$ awk '{sum += $4} END {print sum}' employees.txt
258000
Shiva@DESKTOP-MRCVEKT:~/class8$ awk '$4 > max {max = $4; name = $2} END {print name, max}' employees.txt
Rahul 60000
Shiva@DESKTOP-MRCVEKT:~/class8$ sort -k4 -n employees.txt
102 Priya HR 45000
105 Sneha HR 48000
101 Arjun IT 50000
104 Vikram Finance 55000
103 Rahul IT 60000
Shiva@DESKTOP-MRCVEKT:~/class8$
```

Screenshot 5: the total of the salaries, the highest earner, and the same answer by sorting.

Commands I used

Command	What it does
awk '{print \$NF}' file	Prints the last column of every line.
awk '{print \$(NF-1)}' file	Prints the column before the last one.
grep -rl "ABC" .	File names of all files containing ABC.
free	Memory usage in kilobytes.
free -h	The same numbers in GB and MB.
awk '\$3 == "IT" {print \$2, \$4}' f	Name and salary of the IT people.
awk '\$4 > 50000 {print \$2, \$4}' f	Name and salary of those earning more than 50000.
awk '{sum += \$4} END {print sum}' f	Adds up the salary column.
awk '\$4 > max {max = \$4; name = \$2} END {print name, max}' f	Name and salary of the highest earner.
sort -k4 -n file	Sorts the file by the 4th column as numbers.